

BIG DATA AFRICA SCHOOL 2026 · 6 OCTOBER 2026

Introduction to Probabilistic Programming

Bayesian thinking, and a hierarchical model in NumPyro

Dr Chris Finlay · EPFL

The next hour

Ideas first, then a model, then your questions

0–30 min

Bayesian ideas, from probability to Bayes' rule, priors and sampling, then the hospital model

30–45 min

Notebook: fit and interpret the hierarchical model in NumPyro

45–60 min

Wrap-up, questions and discussion, about your own problems too

A question to keep in mind

Which hospital has the worse complication rate?

Hospital A

25%

2 complications in 8 patients

H01 in the notebook

Hospital B

13%

52 complications in 400 patients

H12 in the notebook

Most people say A. We'll come back to this with a better answer.

Two views of probability

Bayesian probability measures uncertainty

Frequentist

Probability is a long-run frequency. Parameters are fixed; the data are random. Asks: what would happen if I repeated this experiment many times?

Bayesian

Probability is a degree of belief given what you know. Unknown parameters get distributions. Asks: given these data, how plausible is each value?

Both are useful. Bayesian methods shine when data are scarce, structure is rich, or you need uncertainty you can act on.

Probability primer

Three definitions

Joint

$$p(A, B)$$

Marginalisation

$$p(A) = \Sigma p(A, b)$$

$$= p(A, b_1) + p(A, b_2) + \dots + p(A, b_n)$$

$$p(A) = \int p(A, b) db$$

Conditioning

$$p(A \mid B) = p(A, B) / p(B)$$

Probability primer

A joint distribution covers every combination

	Rain	No rain	Total
Cloudy	0.15	0.25	0.40
Clear	0.05	0.55	0.60
Total	0.20	0.80	1.00

$$p(\text{Rain, Cloudy}) = 0.15$$

The probability that **both** happen. The four inner cells cover every possibility, so they sum to 1.

Two operations get you everything else: **marginalisation** and **conditioning**.

Marginalisation

Sum out what you don't care about

	Rain	No rain	Total
Cloudy	0.15	0.25	0.40
Clear	0.05	0.55	0.60
Total	0.20	0.80	1.00

$$p(A) = \sum p(A, b)$$

$$p(\text{Rain}) = 0.15 + 0.05 = 0.20$$

Sum over every value of the thing you don't care about.
Rows work the same way: $p(\text{Cloudy}) = 0.15 + 0.25 = 0.40$. With continuous unknowns, the sum becomes an integral.

Conditioning

Learning something means zooming in

	Rain	No rain	Total
Cloudy	0.15	0.25	0.40
Clear	0.05	0.55	0.60
Total	0.20	0.80	1.00

$$p(A \mid B) = p(A, B) / p(B)$$

$$p(\text{Rain} \mid \text{Cloudy}) = 0.15 / 0.40 = 0.375$$

Keep only the row where B happened, then divide by its marginal so it sums to 1. Seeing clouds raised our belief in rain from 0.20 to 0.375.

Bayes' rule

It follows from conditioning in one line

Conditioning, written both ways

$$p(\theta, y) = p(\theta | y) p(y) = p(y | \theta) p(\theta)$$

$$p(\theta | y) = p(y | \theta) \cdot p(\theta) / p(y)$$

Posterior

What we believe after seeing the data

Likelihood

How probable the data are, for each value of θ

Prior

What we believed before

Evidence

$p(y) = \sum p(y | \theta) p(\theta)$: a marginalisation, and the hard part

Bayes in action

A positive test: how worried should you be?

Prevalence

the prior · $P(D)$

0.1%

of people have the disease: 1
in 1,000

True positive rate

sensitivity · $P(+ | D)$

99%

of sick people test positive

False positive rate

$1 - \text{specificity} \cdot P(+ | \text{no } D)$

1%

of healthy people also test
positive

You test positive. What is $P(D | +)$? Take a guess.
The test is 99% accurate either way.

Bayes' rule, step by step

Plug the numbers in

$$P(D \mid +) = P(+ \mid D) \cdot P(D) / P(+)$$

1 Numerator

true positive rate $\times$ prevalence

$$0.99 \times 0.001 = 0.00099$$

2 Evidence $P(+)$

marginalise over D

$$\begin{aligned} &P(+ \mid D) P(D) + P(+ \mid \text{no } D) P(\text{no } D) \\ &= 0.00099 + 0.01 \times 0.999 = 0.00099 + 0.00999 = 0.01098 \end{aligned}$$

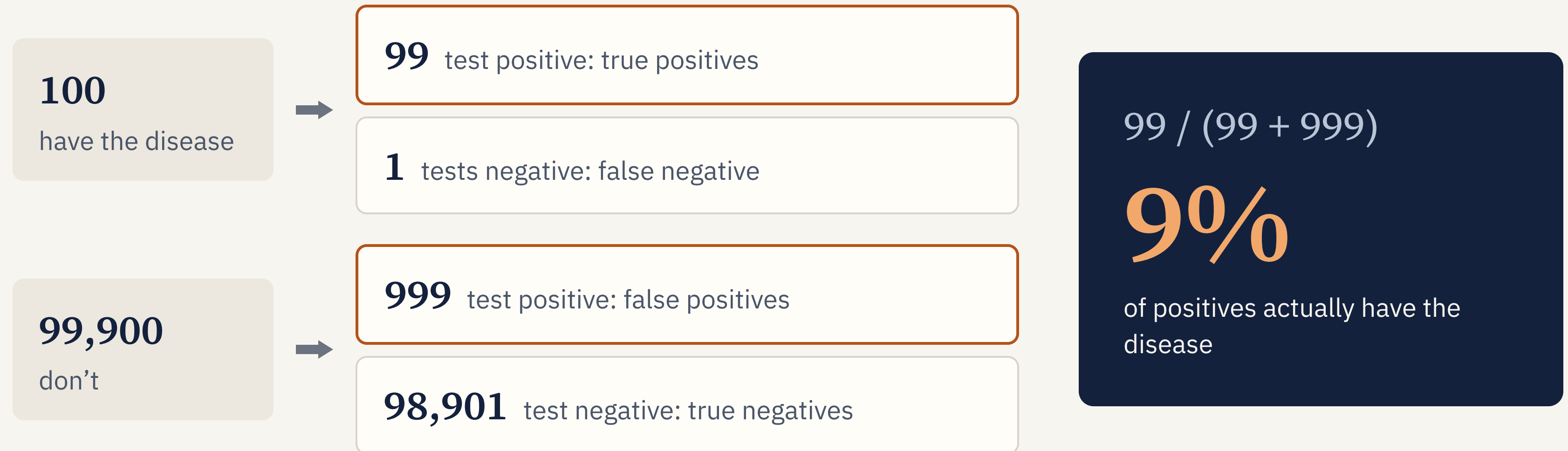
3 Divide

the posterior

$$P(D \mid +) = 0.00099 / 0.01098 \approx 0.090 \approx 9\%$$

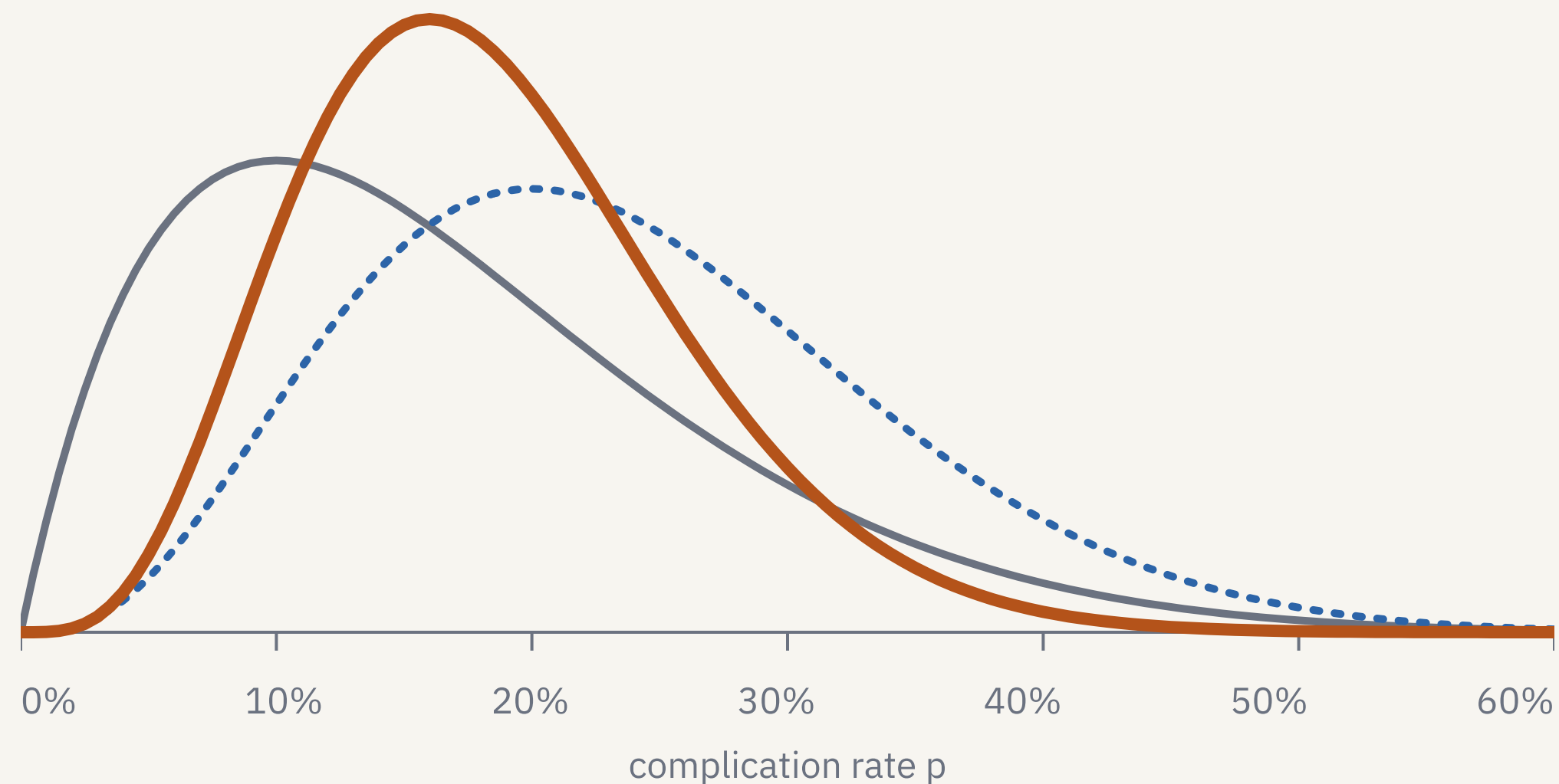
The same answer by counting

Imagine 100,000 people



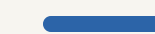
From prior to posterior

One hospital: 3 complications in 15 patients



Prior Beta(2, 10)

Complications are uncommon



Likelihood

The data alone point to $3/15 = 20\%$



Posterior Beta(5, 22)

A compromise, and narrower than either

Priors

Choosing a prior is part of building the model

Encode what you know

Surgical complications are uncommon, so put most belief below 50%

Rule out the absurd

Weakly informative priors exclude impossible values, not plausible ones

Check by simulating

Prior predictive check: generate fake data from the prior. Does it look sane?

Priors are explicit and open to criticism. With enough data, the likelihood dominates.

Computation

Why the denominator is hard

$$p(\theta \mid y) = p(y \mid \theta) \cdot p(\theta) / p(y)$$

Top: easy

Likelihood $\times$ prior: one quick calculation for any θ

Bottom: hard

$p(y) = \int p(y \mid \theta) p(\theta) d\theta$ adds up over every possible θ at once

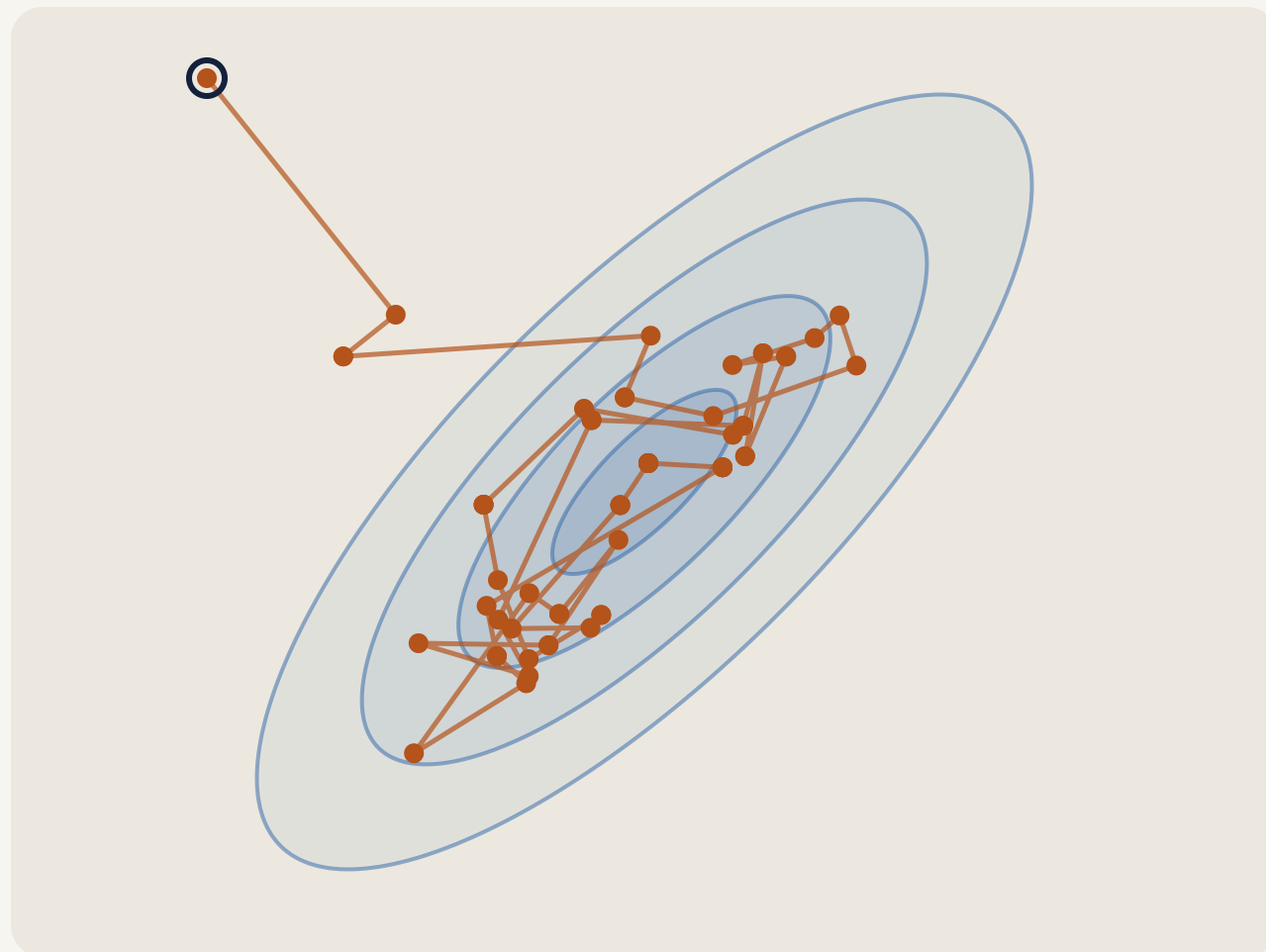
Disease test: $p(+)$ was a sum of 2 terms.

A model with 14 unknowns and a grid of 100 values each:

10^{28} evaluations of the top

Computation

MCMC never needs the denominator



A chain exploring a two-parameter posterior

Propose a move from θ to θ' and compare:

$$\frac{p(\theta' | y)}{p(\theta | y)} \\ = \frac{p(y | \theta') p(\theta')}{p(y | \theta) p(\theta)}$$

$p(y)$ cancels: never computed

Always step uphill, sometimes step downhill. Repeat thousands of times: the points visited are samples from the posterior.

Then just count: $P(\text{rate} > 20\%) \approx$ the fraction of samples above 20%.

Diagnostics

Check the sampler before trusting the answer

R-hat ≤ 1.01

Run several chains from different starts. Do they agree?

ESS in the hundreds

Effective sample size: how many independent draws your correlated samples are worth

Zero divergences

Divergences flag regions the sampler couldn't follow, so results may be biased

Probabilistic programming

A model is a program that could generate your data

```
def one_hospital(n, y=None):  
    p = numpyro.sample("p", dist.Beta(2, 10))  
    numpyro.sample("y", dist.Binomial(n, probs=p), obs=y)  
  
mcmc = MCMC(NUTS(one_hospital), num_samples=1000)  
mcmc.run(key, n=15, y=3)  
p_samples = mcmc.get_samples()["p"]
```

You write the generative story.

The engine does the inference.

Leave out `y` and the same function simulates fake data.

Stan · PyMC · NumPyro · Pyro · Turing.jl

Prior

sample without obs: an unknown, with what we believe before the data

Likelihood

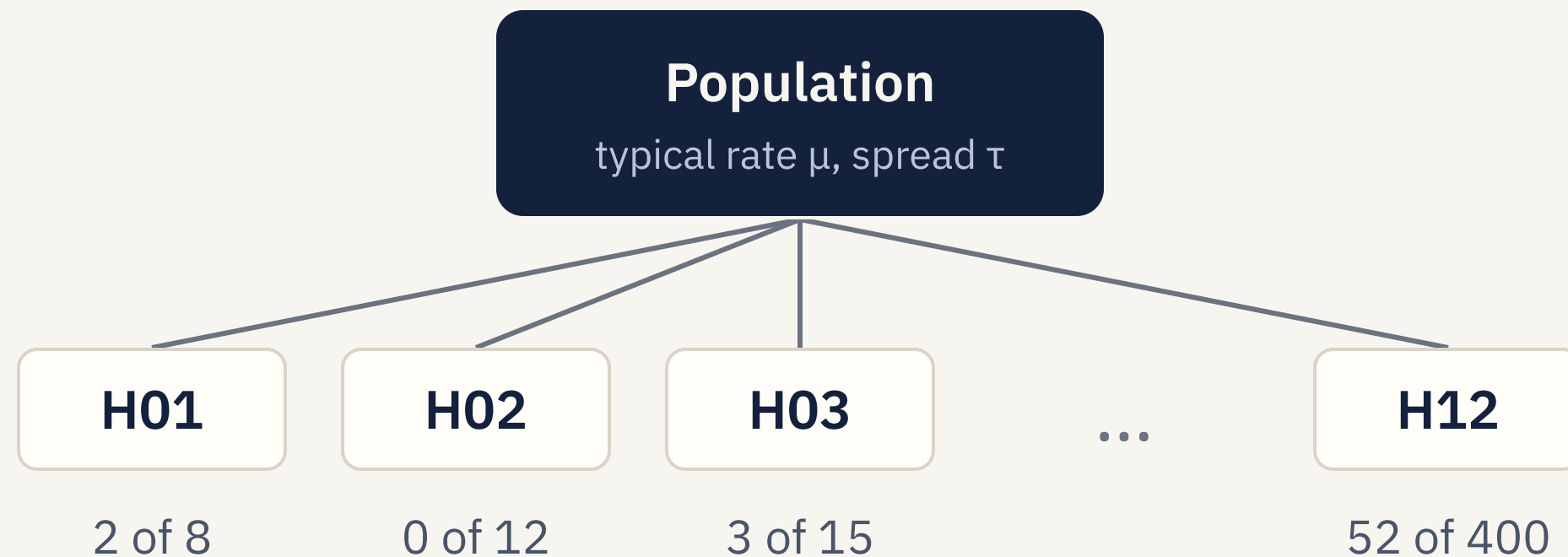
`obs=y` ties the variable to the data. Here `n` = patients treated, `y` = complications

Inference

NUTS turns prior + likelihood into posterior samples

Hierarchical models

When your data come in groups



Hospitals differ, but they are related: learning about one tells you something about the others.

The model learns the population too. How much hospitals differ (τ) decides how much they share.

Also called **multilevel** or **mixed-effects** models. The same structure appears everywhere: patients in clinics, cells in subjects, students in schools, repeated measurements in people.

Three ways to model groups

How much should hospitals share information?

Complete pooling

Every hospital is the same. One rate. **Ignores real differences.**

No pooling

Every hospital is on its own. **Small hospitals get wild estimates.**

Partial pooling

Different, but related. **The data decide how much to share.**

Partial pooling is the hierarchical (multilevel, mixed-effects) model.

The hierarchical model

Hospital rates come from a population we also learn

$$\mu \sim \text{Normal}(-1.5, 1)$$

the typical hospital (log-odds)

$$\tau \sim \text{HalfNormal}(1)$$

how much hospitals differ

$$\theta_j \sim \text{Normal}(\mu, \tau)$$

hospital j 's own rate (log-odds)

$$y_j \sim \text{Binomial}(n_j, \text{logit}^{-1} \theta_j)$$

y_j complications among n_j patients treated

$\tau \rightarrow 0$ gives complete pooling. $\tau \rightarrow \infty$ gives no pooling. **The model estimates τ from the data.**

In NumPyro

The whole model is eleven lines

```
def partial_pooling_nc(n, y=None):
    mu = numpyro.sample("mu", dist.Normal(-1.5, 1.0))
    tau = numpyro.sample("tau", dist.HalfNormal(1.0))
    with numpyro.plate("hospital", len(n)):
        z = numpyro.sample("z", dist.Normal(0, 1))
        theta = numpyro.deterministic("theta", mu + tau * z)
        numpyro.sample("y", dist.Binomial(n, logits=theta), obs=y)

mcmc = MCMC(NUTS(partial_pooling_nc), num_warmup=1000,
            num_samples=1000, num_chains=4)
mcmc.run(jax.random.PRNGKey(0), n=n_patients, y=complications)
```

Population

μ : typical hospital, τ : spread

Each hospital

$\theta_j = \mu + \tau z_j$, one log-odds per hospital

Likelihood

y_j complications among n_j patients

Inference

NUTS, 4 chains $\times$ 1,000 samples

Bayesian workflow

Fitting is one step of five



In practice you loop: a failed check sends you back to revise the model.

“All models are wrong, but some are useful.”

George E. P. Box, statistician

WALKTHROUGH

Twelve hospitals, one operation

`hierarchical_model_numpyro.ipynb`

What the model found

Back to Hospital A (H01 in the notebook)

0.61

probability that Hospital A is
worse than the typical hospital

Hospital	Patients	Raw rate	P(worse)
Hospital A (H01)	8	25%	0.61
Hospital B (H12)	400	13%	0.56
H09	120	6.7%	0.14
H10	180	7.8%	0.13

$P(\text{worse}) = P(\text{hospital's rate is above the population average} \mid \text{data})$

Before any data, every hospital sits near 0.5. Eight patients barely move it, and even 400 patients at an average rate leave Hospital B near 0.5. Only H09 and H10, large and consistently low, stand out: about 86% likely to be *better* than typical.

What the model found

Partial pooling gets closest to the truth



Average error against the true rates, in percentage points (lower is better)

Partial pooling more than halves the error of raw rates. Small hospitals are pulled toward the population; large ones barely move. We can only score this because the data were simulated.

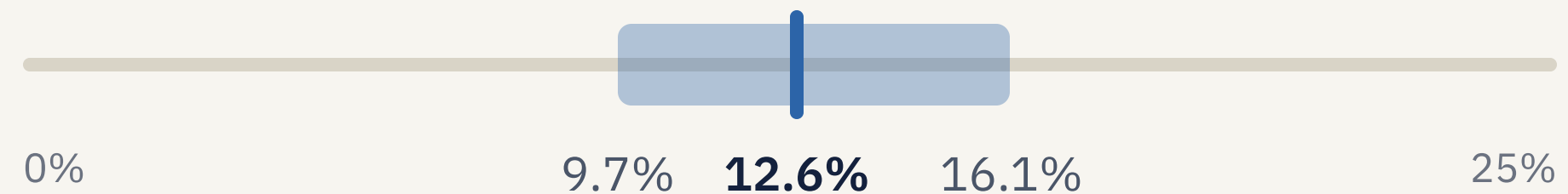
What the model found

It learned how much hospitals differ

$$\tau \approx 0.29$$

spread between hospitals
(true value: 0.30)

In complication rates: a typical hospital is about 12.6%, and most fall within about 10% to 16%.



τ sets how much hospitals share: small τ means pool a lot. The model estimated it from the data, with honest uncertainty (90% interval 0.0 to 0.5).

Takeaways

Five things to remember

1. Bayesian inference turns uncertainty about unknowns into probability distributions
2. A probabilistic program is a generative story; the engine does the inference
3. Hierarchical models share information across groups, and the data decide how much
4. Shrinkage helps: small groups borrow strength and their error drops
5. Follow the workflow: check priors, diagnose the fit, check predictions

Questions?

KEEP GOING

McElreath, *Statistical Rethinking*: book, free lectures, NumPyro port

Gelman et al., *Bayesian Data Analysis* (free PDF)

Gelman et al., “Bayesian Workflow” (2020)

num.pyro.ai: examples gallery

Notebook: github.com/chrisfinlay/probabilistic-programming-tutorial